The AGH University of Krakow

Project Documentation

Sliding Puzzle

Object-Oriented Programming Languages

Electronics 1st year

Author

Norbert Bukała

29.05.2024

1. Project Description

The "4x4 Sliding Puzzle Game" project is an implementation of a logic game in C++. The game consists of arranging shuffled tiles on a 4x4 board in the correct order. The goal is to move the tiles so that they are arranged in ascending order from 1 to 15, leaving one empty space in the bottom-right corner of the board.

2. Project description

The "4x4 Sliding Puzzle Game" project is the implementation of a logic game in C++. The game involves arranging scattered tiles on a 4x4 board in the right order. The goal is to move the tiles to arrange them in ascending order from 1 to 15, leaving one empty square in the lower right corner of the board.

3. User Manual

Game rules:

1. The game starts on a 4x4 board with the tiles arranged in a random but solvable order.
2. The user controls the game using the 'w', 'a', 's', and 'd' keys to move the tiles up, left, down, and right. If they want to end the game early, they can exit using the 'q' (quit) key.
3. The goal of the game is to arrange the tiles from 1 to 15 in ascending order, leaving the empty space in the bottom-right corner of the board.
4. The game ends when the tiles are arranged correctly.

Running the game:

1. Compile the program according to the instructions in section 4.
2. Run the compiled program.
3. Follow the instructions displayed on the screen.

Example of the game in action – terminal view

```
Sliding Puzzle 4x4 Game!
Use
'w'(town)
'a'(left)
's'(down)
'd'(right)
to move the tiles.
Press 'q' to quit the game.
7  3  11  10
4  8  15  13
14  6  5  2
12  1  9
Move (w/a/s/d) or quit (q): w
7  3  11  10
4  8  15  13
14  6  5
12  1  9  2
```

4. Compilation

To compile the project, simply use a standard C++ compiler. The project works on operating systems that support a standard C++ compiler, such as Windows, Linux, and macOS.

Compilation using MinGW:

1. Open cmd (Command Prompt) or PowerShell.
2. Navigate to the directory containing the source files: cd C:\path\to\project.
3. Enter the command: g++ -o SlidingPuzzleGame main.cpp.
4. Run the compiled program: .\SlidingPuzzleGame.

5. Source Files

The project consists of the following source file:

- main.cpp – contains the main program function and the implementation of the Board class.

6. Dependencies

The project does not use any additional external libraries. It relies only on the standard STL library – Standard Template Library.

7. Class Description

Board

The Board class represents the game board in the 4x4 Sliding Puzzle. It contains all the necessary functions for managing the game state, displaying the board, moving tiles, and checking whether the puzzle has been solved.

Public methods:

- **Board()**
  - Class constructor, initializes the board in a solvable state.
- **void display() const**
  - Displays the current state of the board. It uses the console to display each number on the board, while the empty space (0) is shown as a blank.
- **bool moveTile(char direction)**
  - Moves a tile in the specified direction ('w', 'a', 's', 'd').
  - Parameter: direction – the direction in which the tile is moved.
  - Returns true if the move was valid, and false if the move was invalid (e.g. moving beyond the board boundaries).
- **bool isSolved() const**
  - Checks whether the puzzle has been solved.
  - Returns true if all tiles are in the correct positions, and false otherwise.

Code snippet:

```cpp
public:
    // Constructor of the Board class
    Board() : board(4, std::vector<int>(4)) {
        initializeBoard();
    }

    // Display the board
    void display() const {
        for (const auto& row : board) {
            for (int tile : row) {
                if (tile == 0) {
                    std::cout << "\t";
                } else {
                    std::cout << tile << "\t";
                }
            }
            std::cout << std::endl;
        }
    }
```

```cpp
    // Move a tile
    bool moveTile(char direction) {
        int newRow = emptyRow, newCol = emptyCol;
        switch (direction) {
            case 'w': newRow--; break;
            case 's': newRow++; break;
            case 'a': newCol--; break;
            case 'd': newCol++; break;
            default: return false;
        }
        if (newRow < 0 || newRow >= 4 || newCol < 0 || newCol >= 4) {
            return false;
        }
        std::swap(board[emptyRow][emptyCol], board[newRow][newCol]);
        emptyRow = newRow;
        emptyCol = newCol;
        return true;
    }

    // Check if the player has won
    bool isSolved() const {
        int correctTile = 1;
        for (int i = 0; i < 4; ++i) {
            for (int j = 0; j < 4; ++j) {
                if (i == 3 && j == 3) return board[i][j] == 0;
                if (board[i][j] != correctTile++) return false;
            }
        }
        return true;
    }
```

8. Resources

No additional resources. The game does not use external files or text data.

9. Further Development and Improvements

The following features could be added in the future:

- A function to count and display the number of moves made by the player.
- An option to choose the difficulty level with different board sizes.
- A graphical user interface (GUI) for a more attractive appearance and interaction.
- A system for saving and loading the game state.

10. Other

None.